Algoritmer & Datastrukturer
14. september 2026

Angående lærebøgerne

Java:

Sektionerne 1.4-1.6 er ikke ret interessante

C++:

Sektionerne 1.4-1.7 er ikke ret interessante

Rekursion – en metode der kalder sig selv

```
public static int f(int x)
{
    if (x == 0)
        return 0;
    else
        return 2*f(x-1) + x*x;
}
```

- Basis
- Gør fremskridt mod basis
- Problemet reduceres på vej ned (del/divide)
- Løsningen skabes på vej tilbage (hersk/conquer)

Om induktionsbeviser

1. Der etableres en base case (fx i = 1) – i tilfældet den rekursive algoritme: den tilstand hvorefter der termineres. Base casen skal naturligvis være korrekt.
2. Designregel: antag at det rekursive trin virker.
3. Dvs. vi antager at det virker for N, og øvelsen går herefter ud på at vise, at det også virker for N+1.
4. Vi etablerer herefter to udtryk, som angiver resultatet for N+1: i det ene erstatter vi blot N med N+1; i det andet tager vi resultatet for N og adderer det som N+1 bidrager med.
5. Hvis de to udtryk kan omskrives, så de er ens, er udsagnet bevist. (EOP)

Induktionsbeviser

$$\sum_{i=1}^{n} i = 1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$$

Hvis vi bruger formlen for n=n+1 (første ligning), (og vi indsætter blot (n+1) fremfor n)

$$\sum_{i=1}^{n+1} i = \frac{(n+1)((n+1)+1)}{2}$$

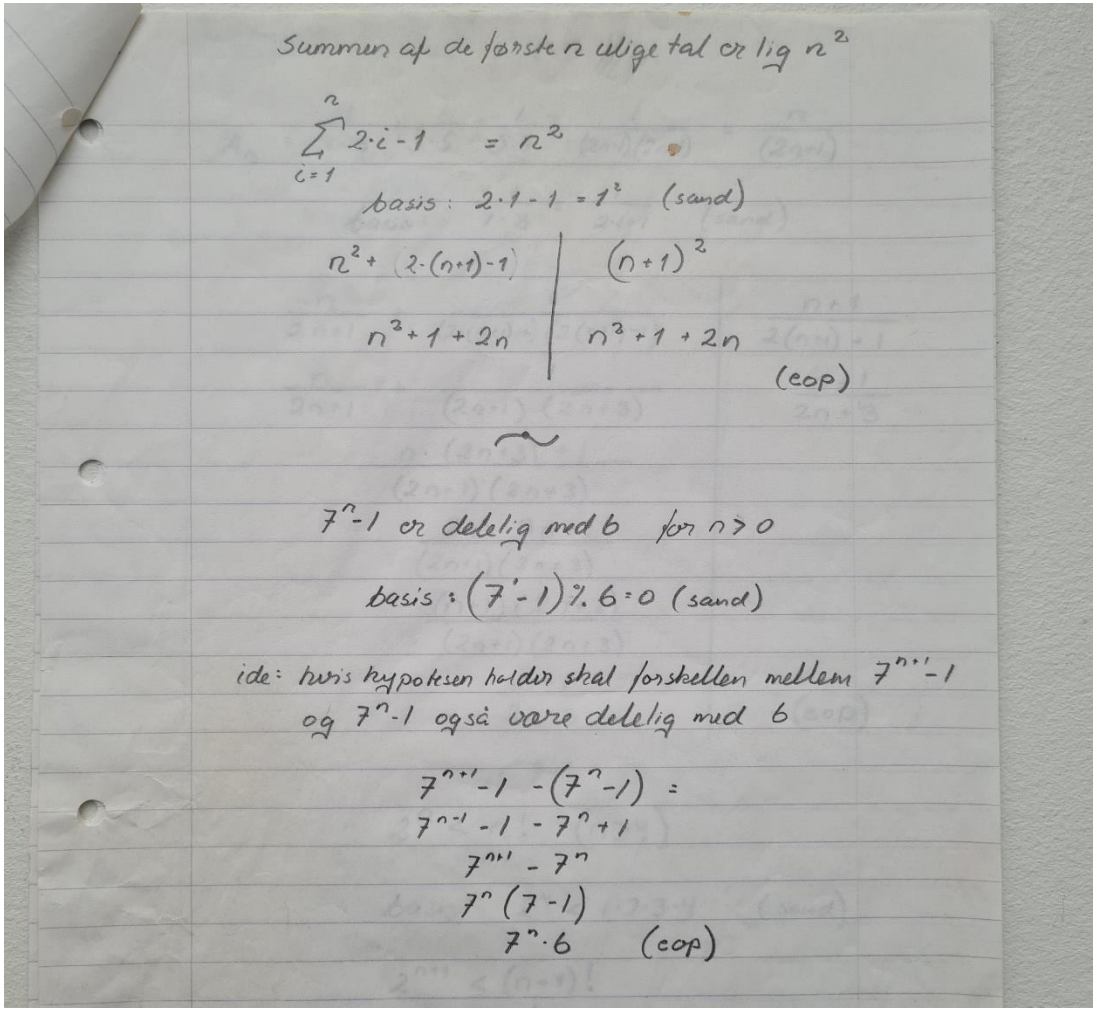
Ved vi at dette nødvendigvis må blive lig følgende (jf. 2. ligning)

$$\frac{n(n+1)}{2} + n + 1$$

Lad os prøve at se om vi kan komme frem til enslydende udtryk:

$$\begin{aligned}
&\sum_{i=1}^{n+1} i = \frac{(n+1)((n+1)+1)}{2} \\
&= \frac{n(n+2) + n + 1 + 1}{2} \\
&= \frac{n(n+1) + n + n + 1 + 1}{2} \\
&= \frac{n(n+1)}{2} + \frac{2n+2}{2} \\
&= \frac{n(n+1)}{2} + (n+1)
\end{aligned}$$

Induktionsbeviser opg. 1 og 5



Summen af de første n ulige tal er lig n^2

$$\sum_{i=1}^{n} 2 \cdot i - 1 = n^2$$

basis: $2 \cdot 1 - 1 = 1^2$ (sand)

$n^2 + (2 \cdot (n+1) - 1)$	$(n+1)^2$
$n^2 + 1 + 2n$	$n^2 + 1 + 2n$

(eop)

$\sim$

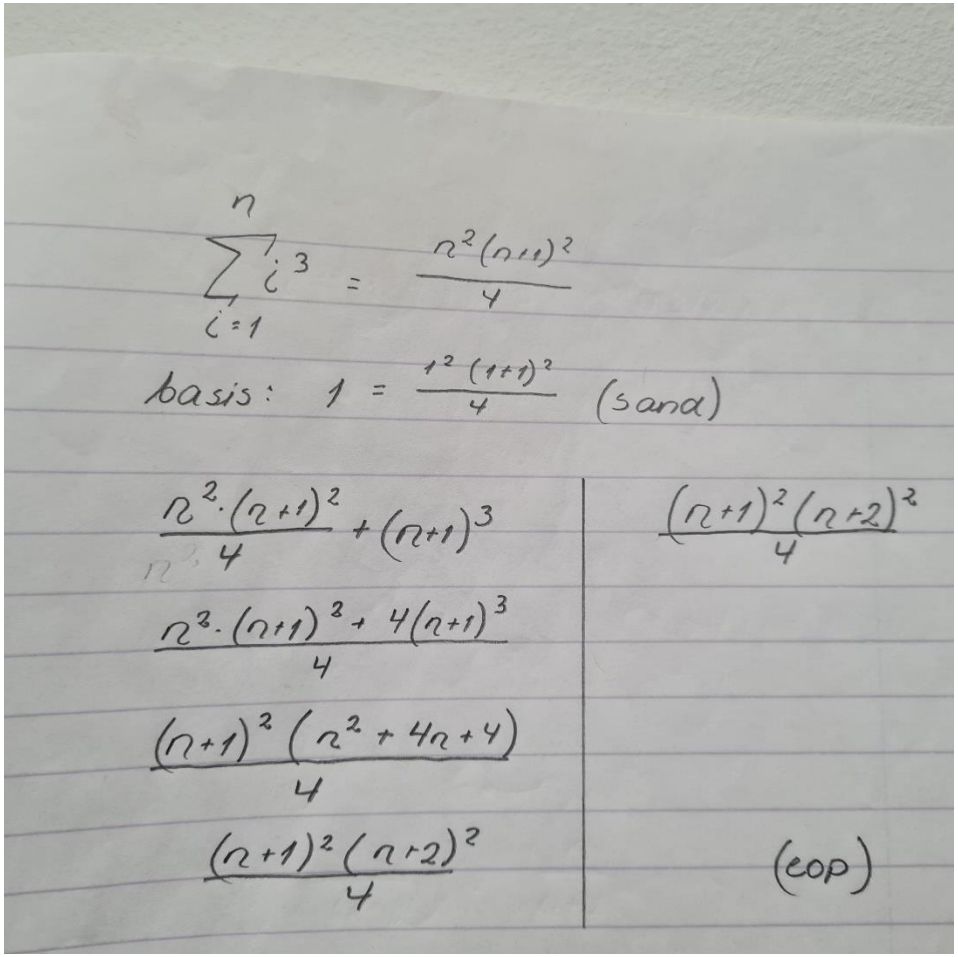
$7^n - 1$ er delelig med 6 for $n > 0$

basis: $(7^1 - 1) \,\%\, 6 = 0$ (sand)

ide: hvis hypotesen holder skal forskellen mellem $7^{n+1} - 1$ og $7^n - 1$ også være delelig med 6

$$7^{n+1} - 1 - (7^n - 1) =$$
$$7^{n-1} - 1 - 7^n + 1$$
$$7^{n+1} - 7^n$$
$$7^n (7 - 1)$$
$$7^n \cdot 6 \quad \text{(eop)}$$

Induktionsbeviser opgave 2



$$\sum_{i=1}^{n} i^3 = \frac{n^2(n+1)^2}{4}$$

basis: $1 = \frac{1^2(1+1)^2}{4}$ (sand)

$\frac{n^2 \cdot (n+1)^2}{4} + (n+1)^3$

$\frac{n^2 \cdot (n+1)^2 + 4(n+1)^3}{4}$

$\frac{(n+1)^2(n^2+4n+4)}{4}$

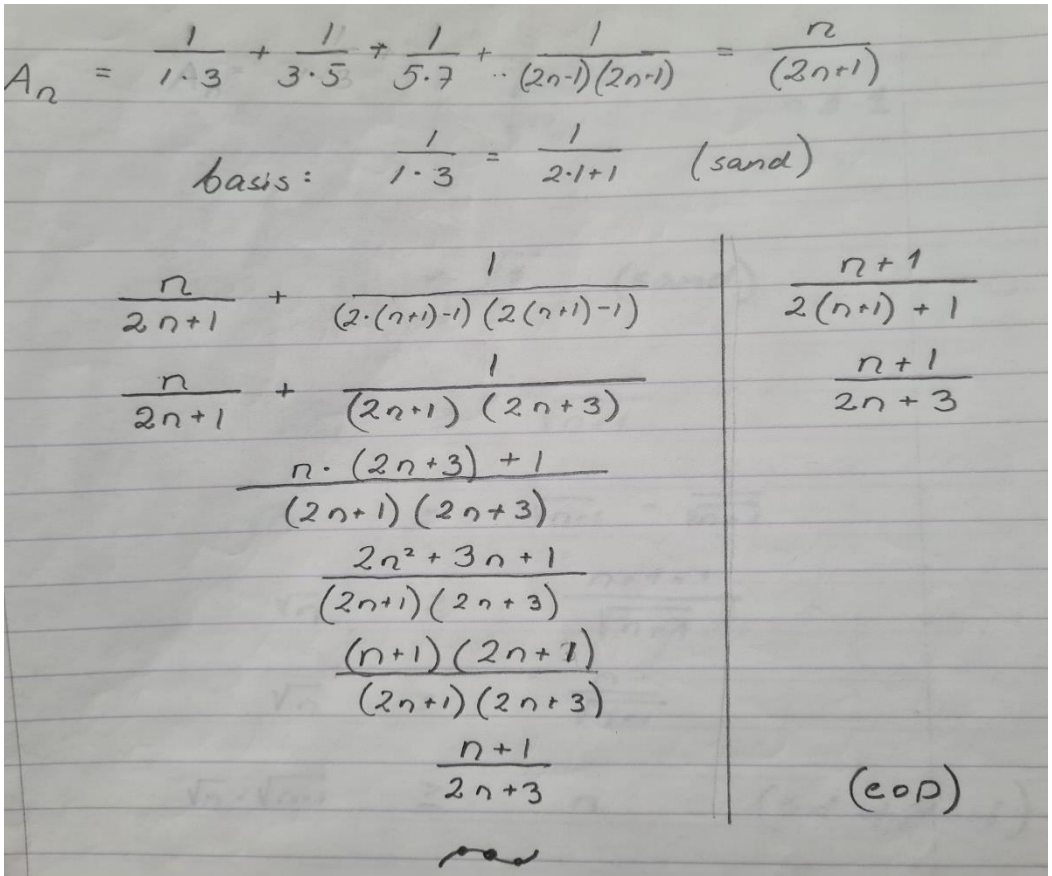
$\frac{(n+1)^2(n+2)^2}{4}$

$\frac{(n+1)^2(n+2)^2}{4}$

(eop)

Induktionsbeviser opgave 3

$$\sum_{1}^{n} \frac{1}{(2n-1)(2n+1)} = \frac{n}{2n+1}$$



$A_n = \frac{1}{1\cdot 3} + \frac{1}{3\cdot 5} + \frac{1}{5\cdot 7} + \cdots \frac{1}{(2n-1)(2n+1)} = \frac{n}{(2n+1)}$

basis: $\frac{1}{1\cdot 3} = \frac{1}{2\cdot 1+1}$ (sand)

$\frac{n}{2n+1} + \frac{1}{(2\cdot(n+1)-1)(2(n+1)-1)}$ | $\frac{n+1}{2(n+1)+1}$

$\frac{n}{2n+1} + \frac{1}{(2n+1)(2n+3)}$ | $\frac{n+1}{2n+3}$

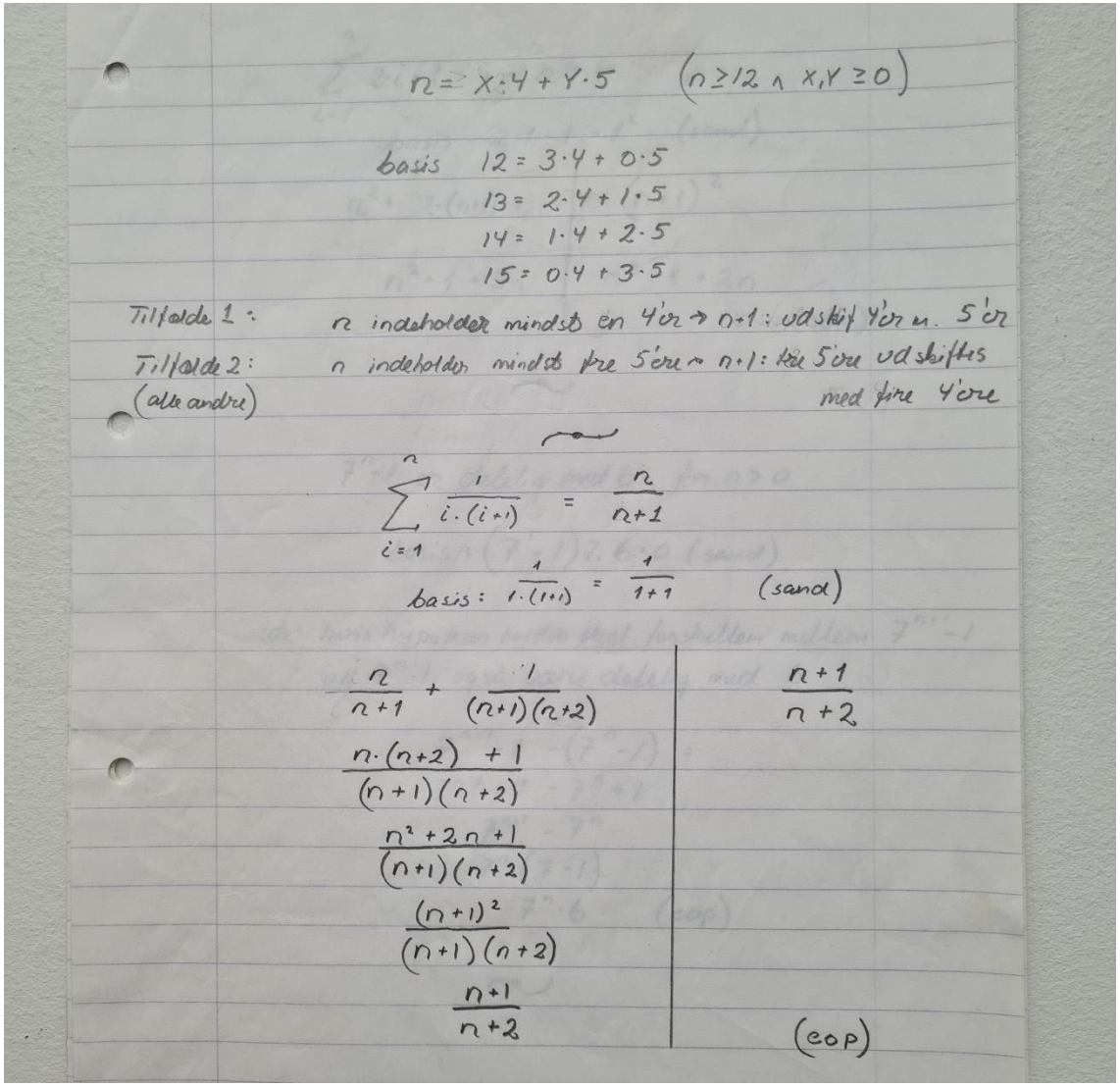
$\frac{n\cdot(2n+3)+1}{(2n+1)(2n+3)}$

$\frac{2n^2+3n+1}{(2n+1)(2n+3)}$

$\frac{(n+1)(2n+1)}{(2n+1)(2n+3)}$

$\frac{n+1}{2n+3}$ (eop)

Induktionsbeviser opg. 4 og ekstra



$n = X \cdot 4 + Y \cdot 5 \quad (n \geq 12 \wedge X, Y \geq 0)$

basis $12 = 3 \cdot 4 + 0 \cdot 5$

$13 = 2 \cdot 4 + 1 \cdot 5$

$14 = 1 \cdot 4 + 2 \cdot 5$

$15 = 0 \cdot 4 + 3 \cdot 5$

Tilfælde 1: n indeholder mindst en 4'er $\rightarrow n+1$: udskift 4'er m. 5'er

Tilfælde 2: (alle andre) n indeholder mindst fire 5'ere $\rightarrow n+1$: fire 5'ere udskiftes med fire 4'ere

$$\sum_{i=1}^{n} \frac{1}{i \cdot (i+1)} = \frac{n}{n+1}$$

basis: $\frac{1}{1 \cdot (1+1)} = \frac{1}{1+1}$ (sand)

$$\frac{n}{n+1} + \frac{1}{(n+1)(n+2)} \qquad \frac{n+1}{n+2}$$

$$\frac{n \cdot (n+2) + 1}{(n+1)(n+2)}$$

$$\frac{n^2 + 2n + 1}{(n+1)(n+2)}$$

$$\frac{(n+1)^2}{(n+1)(n+2)}$$

$$\frac{n+1}{n+2}$$

(eop)

Store O (Big-Oh)

Historien kort fortalt:

Det drejer sig om antallet af *operationer* eller udførte statements. Dem skal vi 'tælle' eller regne ud, og N angiver antallet af elementer, vi behandler.

Den højeste potens af antal operationer udtrykt ved N er afgørende for en algoritmes køretidskompleksitet, fordi alt andet, for store værdier af N, er irrelevant.

essensen af siderne 69-72 (C++) og 49-52 (Java)

Tommelfingerregler

- Halvering af problem O(log N)
- En for-løkke: O(N)
- To for-løkker i sekvens: O(N)
- for-løkke med indb. halvering: O(N log N)
- To nestede for-løkker: $O(N^2)$
- Tre nestede for-løkker: $O(N^3)$
- Tjek alle kombinationer: $O(2^N)$

Store O

$$T(N) = O(f(N))$$
$$hvis\ der\ findes\ positive\ konstanter$$
$$c\ og\ n_{0,}\ således\ at\ T(N) \leq cf(N), når\ N \geq n_0.$$

Eksempel

$$f(N) = N \; og \; T(N) = 10N + 1000$$

$$\mathrm{T}(N) \leq kf(N)$$

$$10N + 1000 \leq kN$$

$$1000 \leq (\mathrm{k} - 10)\mathrm{N}$$

som er sandt, hvis k = 11 og $\mathrm{N} \geq n_0 = 1000$

Vækstrater

Function	Name
c	Constant
$\log N$	Logarithmic
$\log^2 N$	Log-squared
N	Linear
$N \log N$	
N^2	Quadratic
N^3	Cubic
2^N	Exponential

Figure 2.1 Typical growth rates

Vækstrater

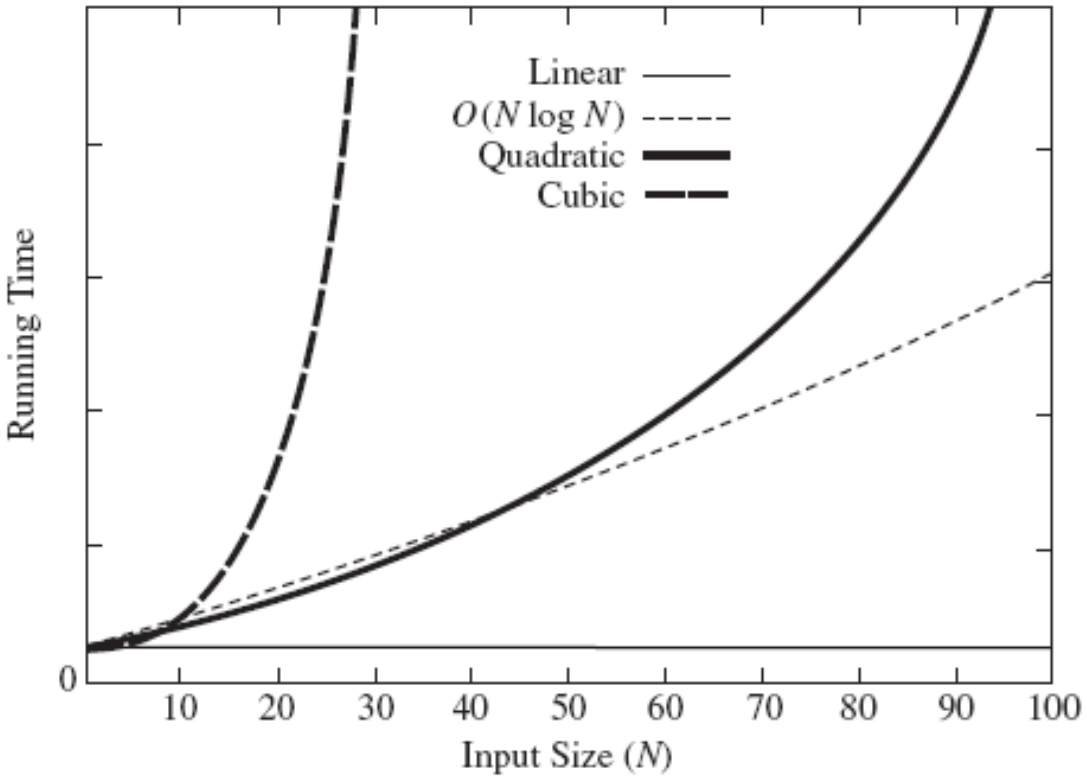


Figure 2.3 Plot (N vs. time) of various algorithms

Store O udtryk indeholder **IKKE** konstanter

	O(N log N)	ok
	O(N)	ok
	O(5N/3+N)	IKKE ok
5N/3+N =>	O(N)	ok

Opgaveeksempel

```
int bigOh(int n)
{
    int x = 0;
    for (int i = 1; i < n; i *= 2)
    {
        for (int j = 1; j <= n; j++)
        {
            for (int k = 1; k <= n; k++)
            {
                x++;
            }
        }
    }
    return x;

}
```

Test af `bigOh(n)`

n	log n	n^2	n^2*log n	x
4	2	16	32	48
16	4	256	1024	1280
64	6	4096	24579	28672
256	8	65536	524288	589824
1024	10	1048576	10485760	11534336

Opgaver i tidskompleksitet (1)

Hvad er Store O tidskompleksiteten af nedenstående metode. Begrund dit svar.

```
public static int myMethod( int[] arr )
{
     int x = 0;
     for (int i = 0; i < arr.length; i++)
          for (int j = 0; j < arr.length/2; j++)
               for (int k = 0; k < arr.length; k++)
               {
                    x++;
                    if (k==1)
                         break;
               }
     return x;
}
```

Opgaver i tidskompleksitet (2)

Hvad er Store O tidskompleksiteten af nedenstående metode. Begrund dit svar.

```
public static int myMethod1( int[] arr )
{
     int x = 0;
     for (int i = 0; i < arr.length/2; i++)
          for (int j = 0; j < arr.length; j++)
              for (int k = 0; k < arr.length; k++)
              {
                  x++;
                  if (k==arr.length/2)
                      break;
              }
     return x;
}
```

The attribute *length* denotes the number of elements in the array.

Opgaver i tidskompleksitet (3)

Hvad er Store O tidskompleksiteten af metoden `func1`. Begrund dit svar.

```
public static int func2(int N)
{
    int res = 0;
    for (int i = 0; i < N; i++)
        res = res + 1;
    return res;
}

public static int func1(int N)
{
    int x = 0;
    for (int i = 0; i < N; i++)
        x = x + func2(N);
    return x;
}
```

Abstrakt Datatype

- ”a set of objects with a set of operations” (p. 95; p. 77).
- Implementering er skjult.
- Operationernes adfærd er et design spørgsmål.
- Forløber for OO.

Simple datastrukturer/datatper

- Stack – push & pop, FILO
- Queue – enqueue & dequeue, FIFO
- List – mange forskellige slags

Stakken

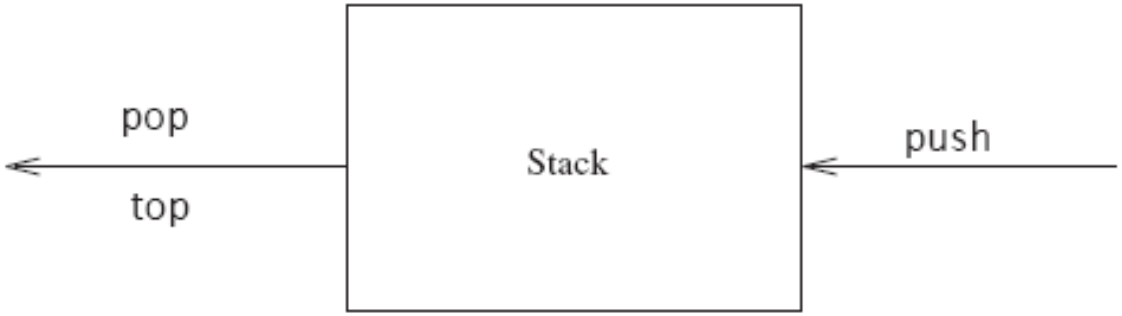


Figure 3.33 Stack model: input to a stack is by `push`, output is by `pop` and `top`

Stakken

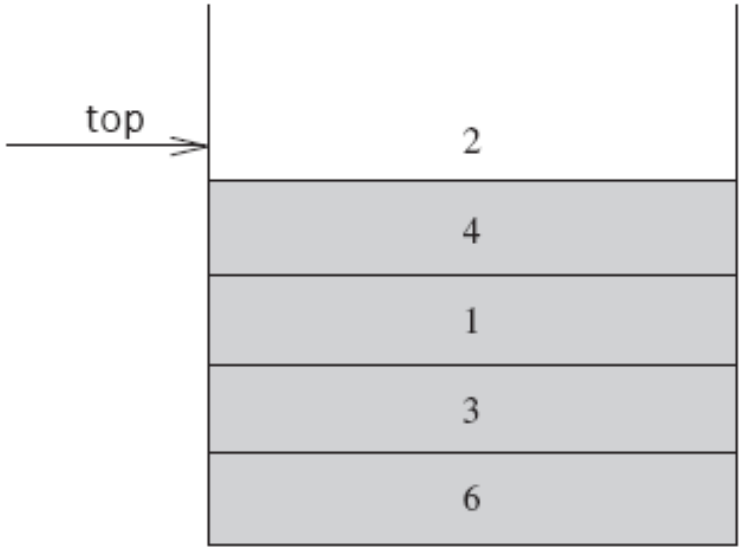


Figure 3.34 Stack model: Only the top element is accessible

Metoder i Stack

Metoden til at tilføje et element til en stak kaldes sædvanligvis *push.*

Metoden til at fjerne et element i en stak kaldes sædvanligvis for *pop*, og den returnerer elementet.

Køen

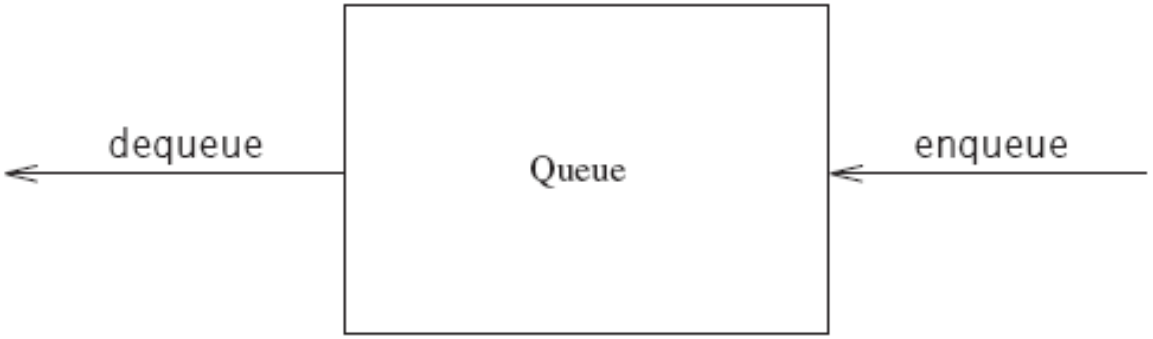
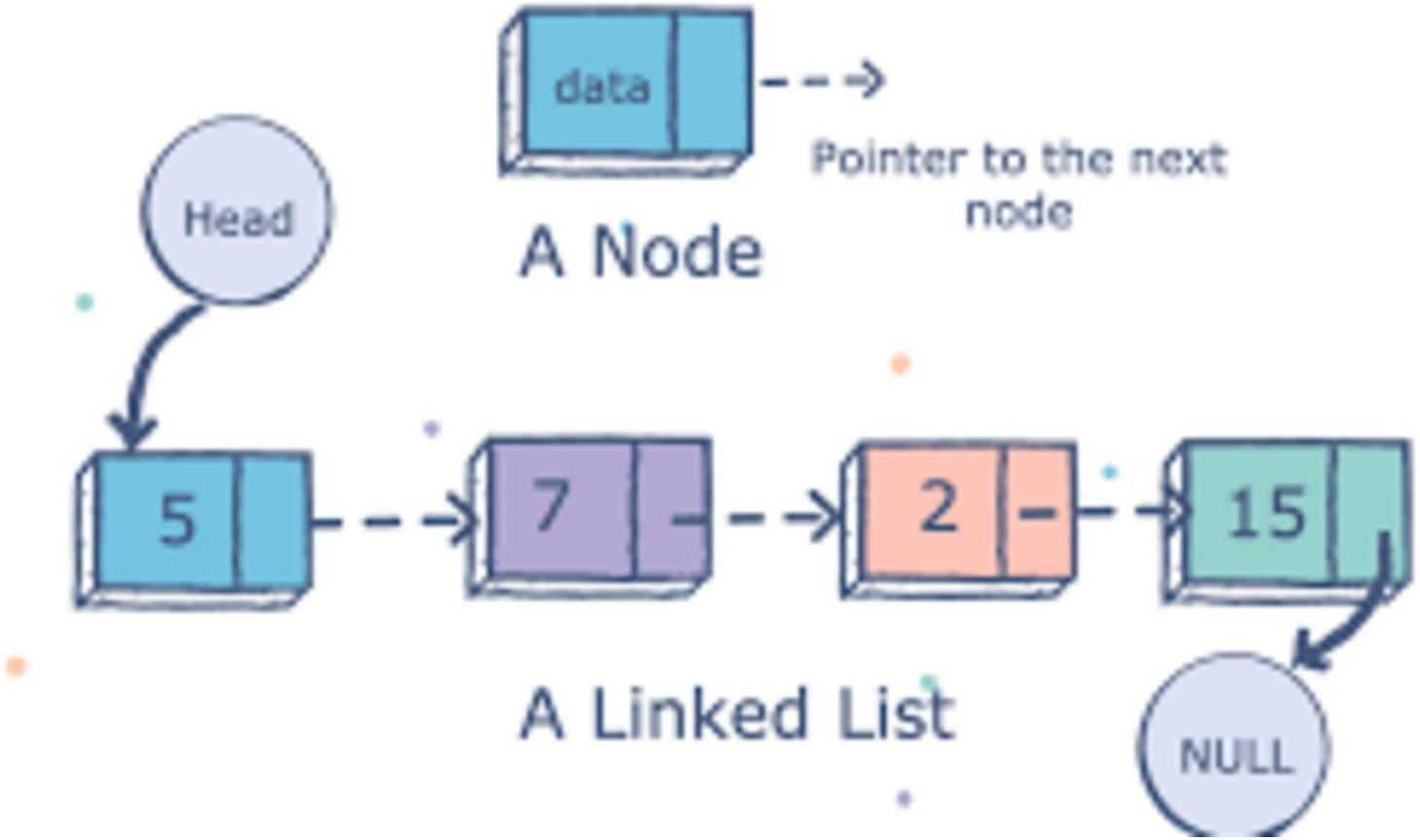


Figure 3.37 Model of a queue

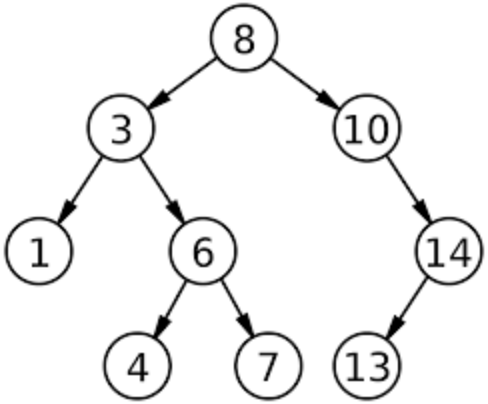
Linked list


data
Pointer to the next node
A Node
Head
5
7
2
15
NULL
A Linked List

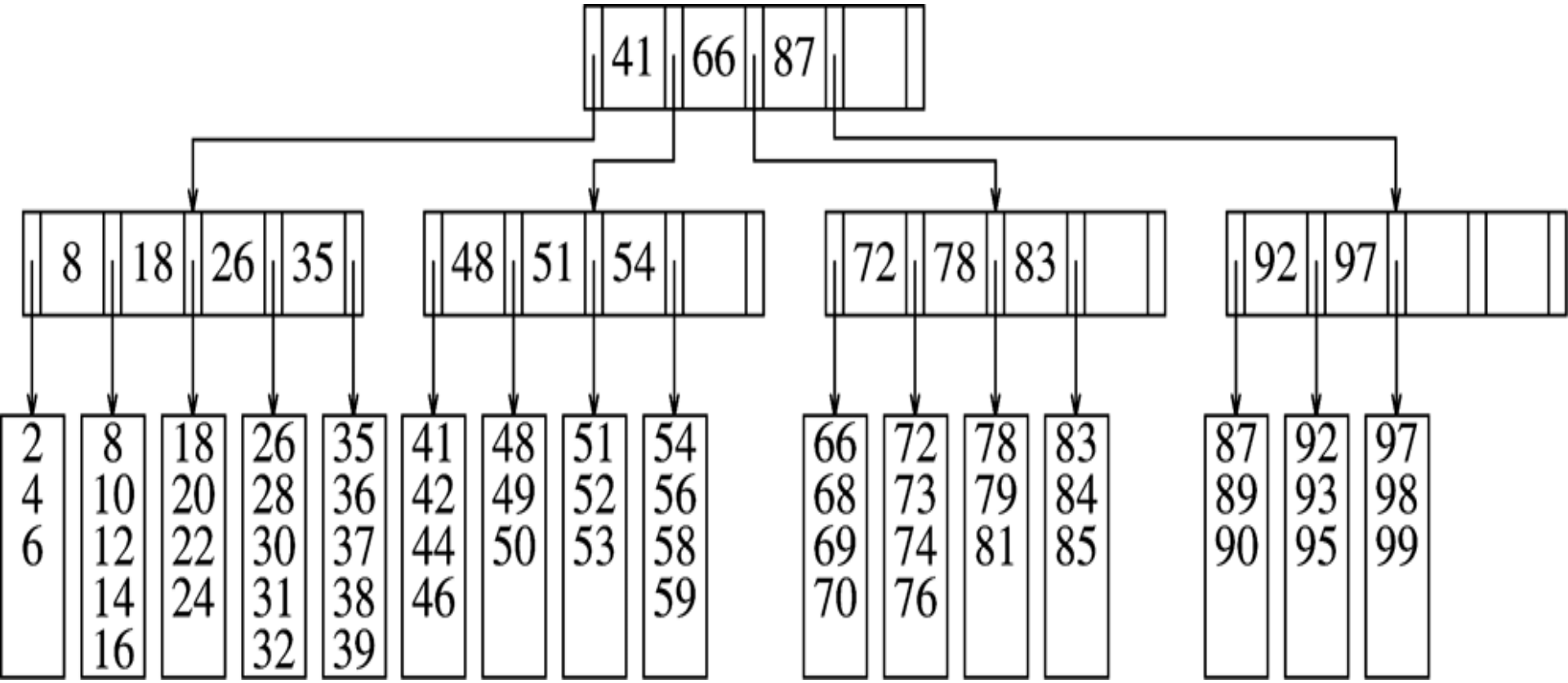
Øvrige emner (kronologisk)

- Hashing
- Prioritetskø
- Sortering
- Træer
- Grafer

Binært søgetræ



5-ordens B-træ



Øvelse (stak)

Implementér din egen stak og skriv dernæst nedenstående metode. Du skal bruge stakken. Tællere må ikke anvendes:

```
boolean balPar(String text);
```

Metoden tjekker om parenteser () i parameteren er balanceret. En udvidet version vil kunne omfatte {} og [].

Øvelse (kø)

Skriv en klasse, som implementerer en cirkulær kø og test den.

En cirkulær kø kan implementeres som et array (med fast størrelse). Når man når til det sidste element, gås videre til det første element, hvis der er plads. Køen kan således løbe fuld.

En udvidet version kunne udvide arrayet, når det løber fuldt.

Øvelse (liste)

Øvelse 3.11 i lærebogen

Find en implementering af linked list på nettet eller skriv din egen.

http://stackoverflow.com/questions/4066729/creating-a-linkedlist-class-from-scratch

Vigtigste slides

6, 7, 12, 15, 16